

Exact Sparse Regression (ℓ_0 norm)

Klaudia Kwoka, Pola Mościcka, Maciej Wach

1 Introduction

The goal of this project is to study the exact sparse regression problem with an ℓ_0 constraint. The objective is to identify a subset of at most k predictors that best explains the response variable in a least squares sense.

This problem is known as *best subset selection* and has been extensively studied in statistics and optimization due to its strong interpretability and predictive performance. However, the ℓ_0 constraint makes the problem computationally challenging, as it requires searching over many possible subsets of variables.

Recent advances in Mixed-Integer Optimization (MIO) have made it possible to solve such problems more effectively in practice [1]. In this work, we focus on the setting where the number of observations exceeds the number of features, i.e., $n > p$.

2 Best subset selection problem

2.1 Problem formulation

Given a response vector $y \in \mathbb{R}^n$ and a feature matrix $X \in \mathbb{R}^{n \times p}$, we aim to find a regression vector $\beta \in \mathbb{R}^p$ with at most k nonzero entries that minimizes the least squares error.

This problem is known as *best subset selection* and is formulated as:

$$\min_{\beta} \frac{1}{2} \|y - X\beta\|_2^2 \quad \text{s.t.} \quad \|\beta\|_0 \leq k. \quad (1)$$

The ℓ_0 norm counts the number of nonzero elements in β .

2.2 Computational complexity

Due to the cardinality constraint, best subset selection involves choosing an optimal subset of variables from an exponentially large number of possible combinations. Because of this combinatorial nature, the problem is NP-hard. In particular, it is noted in [1] that state-of-the-art implementations, such as the R package *leaps*, do not scale to problem sizes larger than $p=30$.

2.3 Lasso

To address the computational challenges of best subset selection, Lasso has been proposed as a convex relaxation of the ℓ_0 -constrained problem. In the linear regression setting, it solves a least squares problem with an added ℓ_1 penalty, which encourages sparsity by shrinking many coefficients exactly to zero.

A key advantage of Lasso is that it results in a convex optimization problem, which makes it computationally efficient and scalable to large datasets. The ℓ_1 penalty leads to a convex objective, which guarantees that any local minimum is also a global minimum. As a result, efficient algorithms such as coordinate descent or proximal gradient methods can solve it reliably even in high dimensions.

Despite its advantages, Lasso has some limitations. In the presence of noise or highly correlated variables, it may include many irrelevant predictors while shrinking their coefficients toward zero, and it

may also fail to select important variables. In addition, it introduces shrinkage bias, since large coefficients are penalized more strongly than smaller ones. As a result, its performance as both a variable selection method and a predictive model can become less reliable in certain settings.

3 Mixed-Integer Optimization Formulation

Best subset selection can also be formulated as a MIO problem by introducing binary variables that indicate whether each predictor is included in the model.

Following [1], we reformulate the problem as a mixed-integer quadratic program. We introduce binary variables $z_j \in \{0, 1\}$ indicating whether feature j is selected.

The model becomes:

$$\min_{\beta, z} \quad \frac{1}{2} \|y - X\beta\|_2^2 \quad (2)$$

$$\text{s.t.} \quad -M_U z_j \leq \beta_j \leq M_U z_j, \quad j = 1, \dots, p, \quad (3)$$

$$\sum_{j=1}^p z_j \leq k, \quad (4)$$

$$z_j \in \{0, 1\}, \quad j = 1, \dots, p. \quad (5)$$

Here, M_U is a sufficiently large constant. If $z_j = 0$, then $\beta_j = 0$, ensuring sparsity.

Unlike Lasso, the MIO formulation is not convex due to the presence of binary decision variables. This makes the problem combinatorial in nature, and in the worst case, solving it may require exploring an exponential number of candidate subsets. As a result, although MIO provides an exact approach to best subset selection, its scalability is not guaranteed for large problem dimensions. Solutions are often obtained within time or optimality gap limits rather than being solved exactly.

4 Choice of the Big-M Parameter

The parameter M_U plays an important role in the MIO formulation. It must be large enough so that the optimal regression coefficients are not incorrectly excluded. In particular, if $\hat{\beta}$ is an optimal solution, we need

$$M_U \geq \|\hat{\beta}\|_\infty. \quad (6)$$

If M_U is too small, the MIO formulation may no longer be equivalent to the original best subset selection problem, because some feasible values of β are artificially removed. On the other hand, if M_U is too large, the continuous relaxation of the MIO problem becomes weak, which may increase the search time. Therefore, the goal is not to choose the largest possible value of M_U , but to choose a valid and reasonably tight upper bound.

4.1 Data-driven bound

One possible approach is to compute coefficient-wise bounds using an upper bound UB on the optimal objective value. For every coefficient β_j , we can solve two auxiliary convex optimization problems:

$$u_j^+ = \max_{\beta} \beta_j \quad \text{s.t.} \quad \frac{1}{2} \|y - X\beta\|_2^2 \leq UB, \quad (7)$$

$$u_j^- = \min_{\beta} \beta_j \quad \text{s.t.} \quad \frac{1}{2} \|y - X\beta\|_2^2 \leq UB, \quad (8)$$

$$UB = \frac{1}{2} \|y - X\beta^{LS}\|_2^2. \quad (9)$$

Then

$$M_j = \max\{|u_j^+|, |u_j^-|\} \quad (10)$$

is an upper bound for $|\beta_j|$, and a global Big-M value can be chosen as

$$M_U = \max_{j=1,\dots,p} M_j. \quad (11)$$

This approach is valid but computationally expensive, because it requires solving two auxiliary optimization problems for each feature.

4.2 Warm-start-based bound

In practice, we use a faster approach based on the warm start solution. First, a first-order hard-thresholding algorithm produces a feasible sparse solution β^{FO} . Then we choose

$$M_U = \tau \|\beta^{FO}\|_\infty, \quad (12)$$

where $\tau > 1$ is a safety multiplier, for example $\tau = 2$.

This choice is practical because β^{FO} is already a feasible sparse solution and gives information about the scale of the regression coefficients. The multiplier τ protects against choosing a bound that is too tight. This method usually gives a much tighter value of M_U than a large arbitrary constant.

5 Warm Start via First-Order Methods

Solving the MIO problem from scratch may be slow, especially when the solver needs to explore many branches before finding a good feasible solution. To improve this, we use a warm start strategy described in [1].

The first-order method is based on iterative hard thresholding. Let

$$g(\beta) = \frac{1}{2} \|y - X\beta\|_2^2. \quad (13)$$

Then the gradient is

$$\nabla g(\beta) = X^T X \beta - X^T y. \quad (14)$$

At iteration t , we first take a gradient step:

$$\beta_m - \frac{1}{L} \nabla g(\beta_m), \quad (15)$$

where

$$L = \lambda_{\max}(X^T X) \quad (16)$$

is a Lipschitz constant of the gradient.

Next, we apply the hard-thresholding operator H_k , which keeps only the k largest coefficients in absolute value and sets the remaining coefficients to zero:

$$\beta_{m+1} \in H_k \left(\beta_m - \frac{1}{L} \nabla g(\beta_m) \right). \quad (17)$$

This produces a feasible solution satisfying $\|\beta_{m+1}\|_0 \leq k$ at every iteration.

5.1 Line search and coefficient polishing

In the implementation, the basic hard-thresholding update is improved using a line search. Instead of moving directly to the thresholded point, we consider a convex combination of the current solution β_m and the thresholded candidate η_m :

$$\beta_{m+1} = \lambda_m \eta_m + (1 - \lambda_m) \beta_m, \quad (18)$$

where

$$\eta_m \in H_k \left(\beta_m - \frac{1}{L} \nabla g(\beta_m) \right), \quad (19)$$

$$\lambda_m = \arg \min_{\lambda \in [0,1]} g(\lambda \eta_m + (1 - \lambda) \beta_m). \quad (20)$$

The parameter λ_m is chosen to minimize the least squares objective along the line segment between the current iterate β_m and the thresholded candidate η_m .

After the support is identified, we additionally polish the coefficients. This means that we fix the selected set of variables

$$S = \{j : \beta_j \neq 0\}$$

and solve an ordinary least squares problem only on the selected columns X_S :

$$\min_{\beta_S} \frac{1}{2} \|y - X_S \beta_S\|_2^2. \quad (21)$$

This step improves the coefficient values without changing the selected subset.

5.2 Stopping criterion

In practice, the first-order warm start method does not need to converge fully. Instead, it is sufficient to stop when further iterations provide only marginal improvement. This is consistent with theoretical arguments in [1], where it is shown that after a relatively small number of iterations, the solution is already close enough to a high-quality support set.

In our implementation, the stopping criterion is:

$$g(\beta_m) - g(\beta_{m+1}) < \varepsilon, \quad (22)$$

where

$$\varepsilon = 10^{-6}. \quad (23)$$

6 Experiments

6.1 Synthetic datasets

Two synthetic data generators were used in the experiments, referred to throughout as **Synthetic Dataset 1** and **Synthetic Dataset 2**. Both generators share the same overall design: a Gaussian matrix X , column-standardized, combined with a sparse true coefficient vector β_0 containing k_0 nonzero entries and additive Gaussian noise calibrated to a target signal-to-noise ratio (SNR). More detailed description of Synthetic Dataset 1 can be found in Section 6.4.

Synthetic Dataset 2 was implemented according to [1] (Section 5.1 Description of Experimental Data), with the only difference being the construction of β_0 : its nonzero coefficients were sampled independently from a Uniform(1, 10) distribution and assigned random positive or negative signs.

Synthetic Dataset 1 was used in Experiment 1 (see Section 6.4), while both Synthetic Dataset 1 and Synthetic Dataset 2 were used in Experiment 3 (see Section 6.6).

6.2 Real dataset

We conduct Experiments 2 and 4 (see Sections 6.5 and 6.7) on an expanded version of the Diabetes dataset from `scikit-learn`¹. The original dataset contains $n = 442$ observations with 10 clinical features. To enrich the feature space, we construct nonlinear transformations of the original variables by including the base features, quadratic terms x_j^2 for all features except the binary *sex* variable, and all pairwise interaction terms $x_j x_\ell$ for $j < \ell$, resulting in a higher-dimensional nonlinear representation of the data. This results in a final feature dimension of $p = 64$. From this expanded dataset, we randomly sample $n = 350$ observations using a fixed random seed for reproducibility (seed = 0). All features are standardized prior to optimization. Each column is centered to have zero mean and scaled to unit ℓ_2 norm. Columns with negligible norm (below 10^{-10}) are left unchanged. The response variable is also centered to zero mean.

6.3 General setup

All experiments were set up in Python and the results are on the GitHub² using libraries numpy, scikit-learn, gurobipy, matplotlib, kagglehub, ucimlrepo. The MIO formulations were solved using Gurobi under an academic license, while First Order method and Lasso relied on numpy and scikit-learn. All synthetic experiments were repeated with fixed random seeds to ensure reproducibility, and the same codebase was reused across the synthetic and real-data experiments to keep the comparison between methods consistent.

6.4 Experiment 1

This experiment compares the following methods:

- Lasso,
- first-order hard-thresholding method,
- MIO with warm start,
- MIO with cold start (MIO without warm start).

The reported results are averaged over 5 repetitions of experiment. This reduces the influence of a single random data generation and gives a more stable comparison of the methods. The synthetic data (Synthetic Dataset 1) are generated from a linear model

$$y = X\beta^0 + \varepsilon, \quad (24)$$

where β^0 is sparse and ε is Gaussian noise. The feature matrix X is generated with correlated predictors, controlled by a correlation parameter ρ . Larger values of ρ make the problem harder because correlated variables are more difficult to distinguish.

Following [1] we use signal-to-noise ratio that controls the noise level in the generated data, defined as

$$\text{SNR} = \frac{\text{Var}(x^T \beta^0)}{\sigma^2}. \quad (25)$$

In this experiment, we use $\text{SNR} = 3$, which means that the signal is three times stronger than the noise.

For every generated dataset, the sparsity parameter k is selected using cross-validation. We consider a set of candidate values of k , train the model for each candidate, and choose the value that gives the lowest validation error. This procedure is used to make the comparison more realistic, since in practice the true sparsity level is usually unknown. Following [1], we evaluate predictive performance using the prediction error defined as

$$\text{PE}(\hat{\beta}) = \frac{\|X\hat{\beta} - X\beta^0\|_2^2}{\|X\beta^0\|_2^2}. \quad (26)$$

¹<https://archive.ics.uci.edu/dataset/34/diabetes>

²<https://github.com/polmos02/OptimizationMIO/tree/main>

This measure compares the predictions obtained from the estimated coefficients $\hat{\beta}$ with the predictions generated by the true sparse coefficient vector β^0 . Lower values of prediction error indicate better predictive performance. The comparison is based on objective value, prediction error, number of selected variables and computation time.

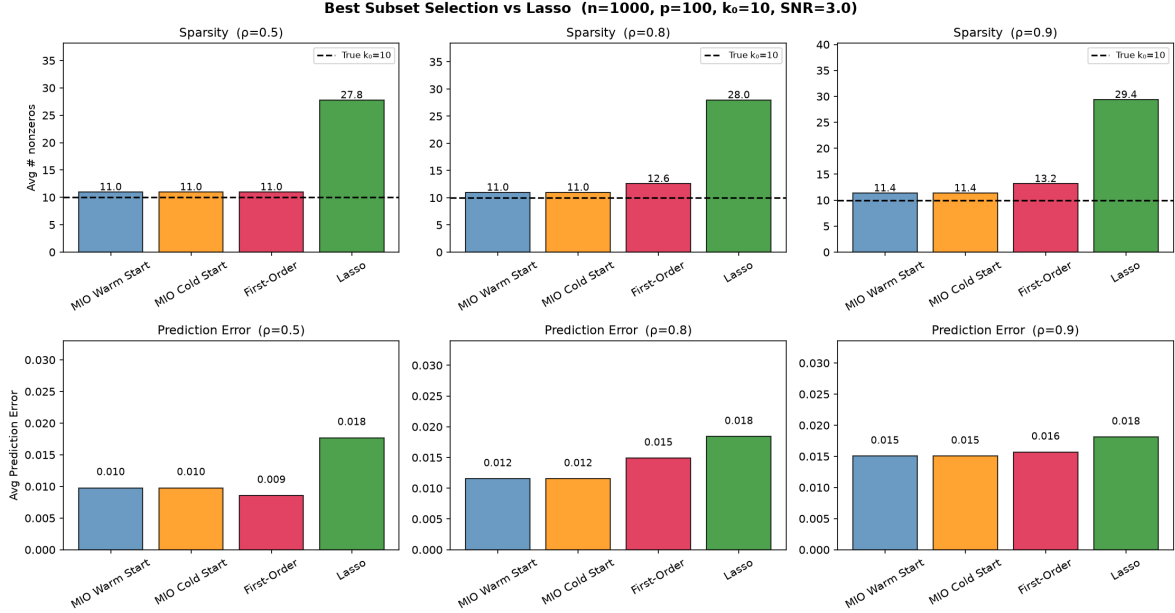


Figure 1: Comparison of MIO, first-order hard-thresholding, and Lasso for different correlation levels.

Figure 1 compares performance of MIO with and without warm start, the first order method and Lasso on synthetic data (see 6.2) with $n = 1000$, $p = 100$, true sparsity $k_0 = 10$ and signal to noise ratio equal to 3.0. The experiment was repeated for three levels of correlation between predictors $\rho = 0.5$, $\rho = 0.8$ and $\rho = 0.9$. The first row of the figure shows the average number of selected variables. The dashed horizontal line corresponds to the true number of nonzero coefficients.

For $\rho = 0.5$, MIO (both warm start and cold start) and the first-order method all select 11 variables on average, closely matching the true sparsity level $k_0 = 10$. As correlation increases, MIO remains close to the desired sparsity: it selects 11 variables for $\rho = 0.8$ and 11.4 variables for $\rho = 0.9$. The first-order method becomes less stable as correlation increases, selecting 12.6 and 13.2 variables on average for $\rho = 0.8$ and $\rho = 0.9$, respectively. Lasso selects many more variables in all cases: 27.8, 28.0, and 29.4 variables on average. This confirms that Lasso does not directly control the number of selected predictors and tends to produce less sparse models.

The second row presents the average prediction error. For $\rho = 0.5$, MIO methods and the first-order method achieve the lowest prediction error, all approximately 0.009–0.010, while Lasso has a much higher error of 0.018. For $\rho = 0.8$, MIO obtains the best prediction error, equal to 0.012, compared with 0.015 for the first-order method and 0.018 for Lasso. For the strongest correlation, $\rho = 0.9$, MIO again gives the lowest prediction error, equal to 0.015, followed closely by the first-order method at 0.016, while Lasso remains highest at 0.018 despite selecting substantially more variables than the true model.

Overall, the results show that MIO provides the best balance between sparsity and prediction accuracy. It selects a number of variables close to the true sparsity level and achieves the lowest prediction error in all three correlation settings. The first-order method is competitive when predictors are weakly or moderately correlated, but its performance deteriorates when correlation is high. Lasso is computationally attractive, but it tends to select too many variables and therefore gives less interpretable models, while also incurring the highest prediction error across all settings.

Table 1 summarizes prediction error, sparsity, and runtime across methods and correlation levels. MIO Warm Start and Cold Start result in identical accuracy and sparsity, as both solve the same opti-

Method	ρ	Avg Pred. Err.	Avg NNZ	Avg Time (s)
MIO Warm Start	0.5	0.0098	11.0	74.5
MIO Cold Start	0.5	0.0098	11.0	91.5
First-Order	0.5	0.0086	11.0	1.7
Lasso	0.5	0.0177	27.8	0.1
MIO Warm Start	0.8	0.0116	11.0	503.2
MIO Cold Start	0.8	0.0116	11.0	1180.3
First-Order	0.8	0.0149	12.6	3.7
Lasso	0.8	0.0184	28.0	0.0
MIO Warm Start	0.9	0.0150	11.4	5686.3
MIO Cold Start	0.9	0.0150	11.4	7685.1
First-Order	0.9	0.0157	13.2	556.2
Lasso	0.9	0.0181	29.4	0.1

Table 1: Performance comparison across methods and correlation levels ρ .

mization problem to but warm starting reduces runtime especially at higher correlations ($\rho = 0.8, 0.9$). MIO’s computational cost grows with ρ , from under 100 seconds at $\rho = 0.5$ to nearly 5700–7700 seconds at $\rho = 0.9$, showing the increasing difficulty of the combinatorial problem under collinearity. The first-order method is faster than MIO (1.7–556.2 seconds) but its runtime and prediction error both increase with ρ , and it selects more variables than the true model at higher correlations. Lasso is the fastest but selects nearly three times as many variables as the truth and has the highest prediction error in two of three settings. Overall, the results illustrate a clear trade-off: MIO offers the best accuracy and sparsity at a steep computational cost, the first-order method provides a reasonable middle ground, and Lasso trades statistical performance for speed.

6.5 Experiment 2

We conduct this experiment using an expanded version of the Diabetes dataset from `scikit-learn` (see Section 6.2).

For each sparsity level k , we solve MIO with warm start. The MIO solver is executed with a time limit of $T = 120$ seconds. We evaluate the method across the following sparsity levels:

$$k \in \{9, 20, 31, 42\}.$$

These values cover low, medium, and high sparsity regimes. To evaluate convergence over time, we define the *relative accuracy* for a given sparsity level k as:

$$\text{RelAcc}_k(t) = \frac{f_k(t) - f_k^*}{f_k^*},$$

where $f_k(t)$ denotes the best objective value found up to time t , and f_k^* is the best objective value obtained within the time horizon T .

This metric measures the relative suboptimality of the incumbent solution over time. For each value of k , the discrete first-order method is first run to obtain an initial solution, which is used to solve the MIO problem with a time limit of $T = 120$ seconds. Objective values are recorded during the optimization process and used to construct a step-function trajectory of $f_k(t)$ over time. Relative accuracy is then computed using the best observed objective value f_k^* . The results show consistent behavior across all sparsity levels. Most improvements occur early in the optimization process, within the first 20 seconds. After this phase, improvements become less frequent and the objective value stabilizes. Lower values of k converge more quickly due to the reduced combinatorial search space, while higher values of k exhibit slower convergence but achieve better final objective values. Overall, the warm start from the discrete first-order method significantly improves early convergence and provides a strong initialization for the MIO solver.

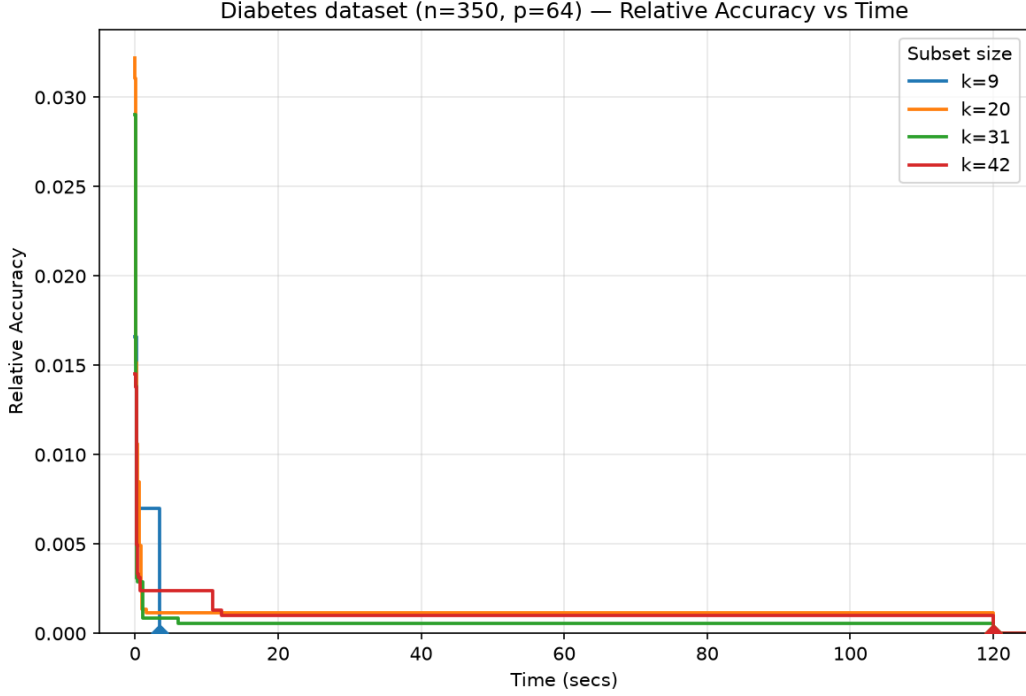


Figure 2: Relative acc

6.6 Experiment 3

6.6.1 Setup

For each of the two synthetic data generators, three controlled experiments were performed, in each of which a single parameter was varied while the remaining parameters were held fixed at $n = 800$, $p = 30$, $\text{SNR} = 3.0$, with true sparsity $k_0 = 10$ and correlation $\rho = 0.5$ throughout. The parameter grids used were:

- $\text{SNR} \in \{0.5, 1, 1.5, 2, 2.5, 3, 3.5, 4.5, 5, 5.5, 6\}$,
- $n \in \{200, 300, 400, 500, 600, 700, 800, 900, 1000, 1200, 1350, 1500, 1600, 1750, 1800, 2000, 2200, 2500\}$,
- $p \in \{10, 20, 30, 40, 50, 60, 70, 80, 90, 100\}$.

Each configuration was repeated 3 times with independent random seeds and reported values are averages over these repeats. For every generated dataset, observations were split into training, validation, and test sets (60%/20%/20%). Three estimation methods were compared:

- **MIO**: solved as a mixed-integer quadratic program (via Gurobi) with warm start
- **First-Order with hard thresholding**,
- **Lasso**: ℓ_1 -regularized regression with the penalty parameter chosen via cross-validation (**LassoCV**).

For MIO and First-Order, the subset size k was selected on the validation set by minimizing validation mean squared error over a candidate range of k values around k_0 . All methods were then evaluated on the held-out test set using two metrics: the **prediction error**, defined as the normalized squared distance between the fitted and true signal and the **sparsity**, defined as the number of nonzero entries in the fitted coefficient vector $\hat{\beta}$.

6.6.2 Results on Synthetic Dataset 1

Three figures 3, 4, 5 illustrate the results of the experiment on the Synthetic Dataset 1.

Varying parameter n : Figure 3.

Error decreases rapidly as the sample size grows past a few hundred observations, with a shared spike around $n \approx 300$ indicating small-sample instability. MIO and First-Order track each other closely across the whole range. Lasso is occasionally a little higher, but the gap narrows and all three methods are nearly indistinguishable once n is large.

MIO and First-Order remain close to 10–13 nonzero coefficients with no clear trend as n grows. Lasso fluctuates between roughly 15 and 22 selected variables, consistently well above the true sparsity level.

Varying parameter p : Figure 4.

As the number of variables grows while k_0 remains fixed at 10, Lasso’s error increases noticeably and becomes the worst-performing method. MIO and First-Order remain substantially lower and close to each other throughout, indicating greater robustness to the addition of irrelevant variables.

The sparsity plot shows clear contrast between methods: Lasso’s selected model size grows approximately linearly with p , exceeding 30 nonzero coefficients at $p = 100$ despite only 10 true active variables. MIO and First-Order remain essentially flat and close to $k_0 = 10$ across the entire range.

Varying parameter SNR: Figure 5.

All three methods improve sharply as SNR increases from 0.5 to about 2, after which the curves flatten close to zero. At very low SNR, MIO performs noticeably worse than Lasso and First-Order, likely because validation-based selection of k becomes unreliable when the signal is weak. Beyond $\text{SNR} \approx 2$, all three methods converge to very similar, low error.

MIO and First-Order select close to the true 10 variables across all SNR levels, staying on or just above the reference k_0 line. Lasso consistently selects around 15 variables, over-selecting relative to the truth regardless of signal strength.

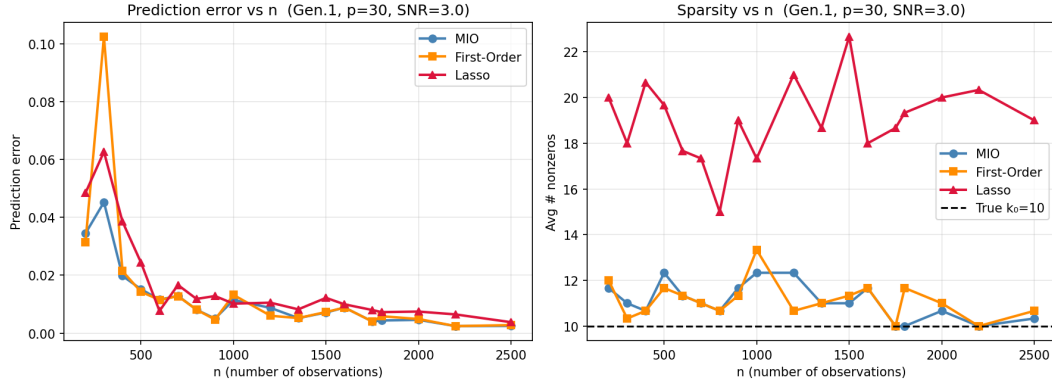


Figure 3: Impact of number of observations n on prediction error and sparsity for Synthetic Dataset1.

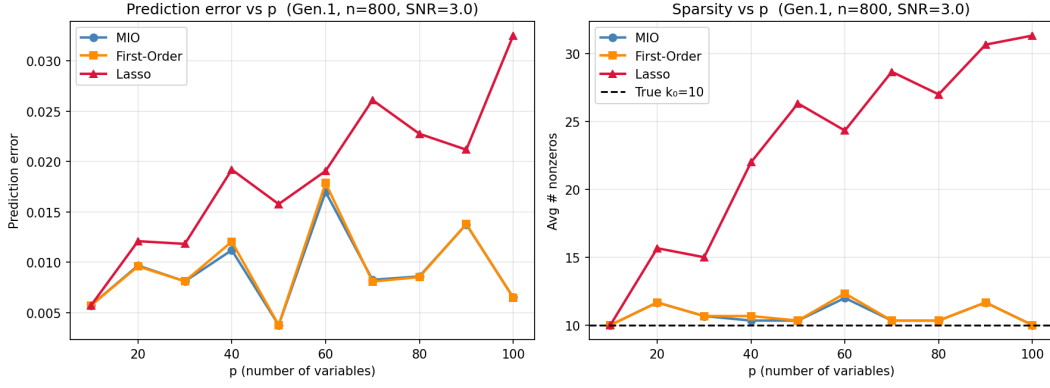


Figure 4: Impact of the number of features p on prediction error and sparsity for Synthetic Dataset1.

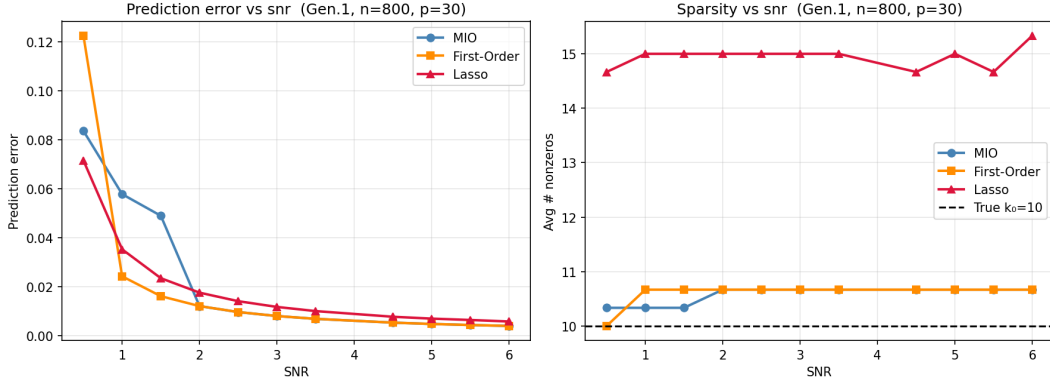


Figure 5: Impact of the SNR value on prediction error and sparsity for Synthetic Dataset1.

6.6.3 Results on Synthetic Dataset 2

Three figures 6, 7, 8 illustrate the results of the experiment on the Synthetic Dataset 2.

Varying parameter n : Figure 6.

A sharp initial drop in prediction error is again observed, with a visible bump around $n \approx 300$ –400 for all methods. MIO and First-Order stay close to each other and generally at or below Lasso, particularly at smaller sample sizes.

For sparsity the pattern resembles Synthetic Dataset 1: MIO and First-Order fluctuate around 8-13 selected variables near the true value, while Lasso is consistently higher and noisier, ranging roughly from 16 to 24.

Varying parameter p : Figure 7.

All three methods perform similarly at small p , with a shared spike in error around $p = 50$ that is bigger for MIO and First-Order than for Lasso. Errors decrease again afterward, and Lasso ends up with higher error than the other two methods at $p = 90$ -100.

As in Synthetic Dataset 1, Lasso's selected model size increases steadily with p , exceeding 30 variables at $p = 100$, while MIO and First-Order remain close to the true value of 10 across the whole range, with only mild fluctuation.

Varying parameter SNR : Figure 8.

The overall prediction error mirrors Synthetic Dataset 1 - rapid improvement up to $SNR \approx 2$, then flattening. Here MIO and First-Order overlap almost exactly throughout, and both remain consistently below Lasso at every SNR level.

MIO and First-Order stay close to the true $k_0 = 10$, settling almost exactly on it from $SNR \approx 2$ onward. Lasso again over-selects substantially, choosing around 18-20 variables across the full SNR range.

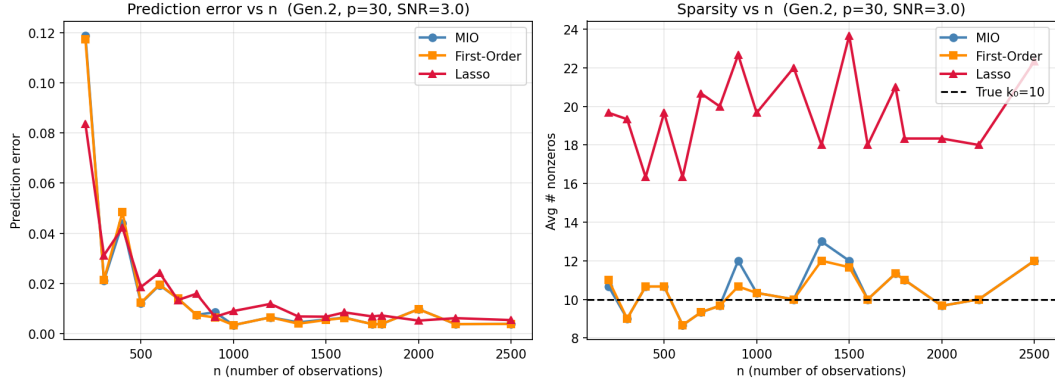


Figure 6: Impact of number of observations n on prediction error and sparsity for Synthetic Dataset2.

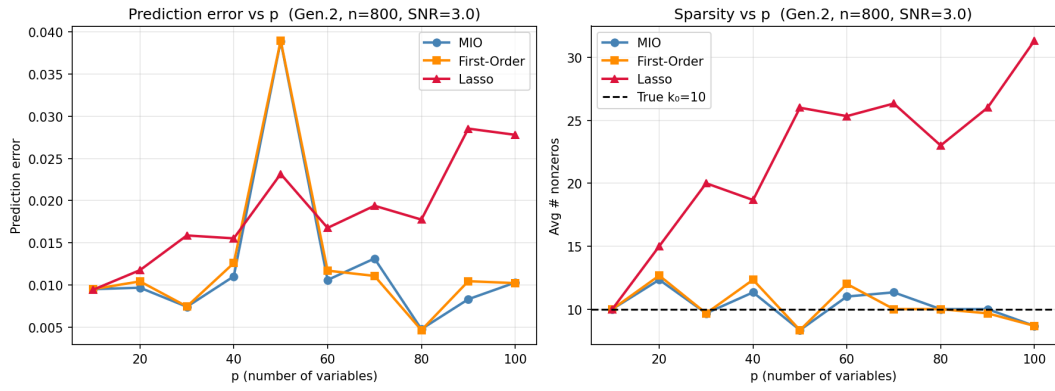


Figure 7: Impact of the number of features p on prediction error and sparsity for Synthetic Dataset2.

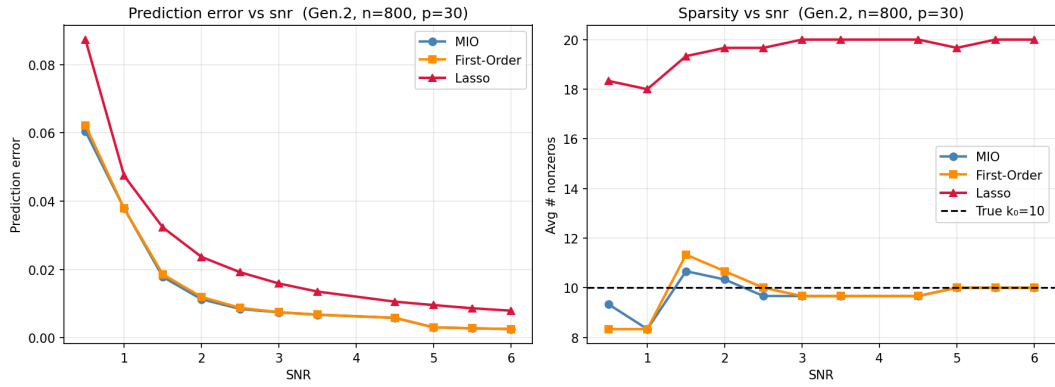


Figure 8: Impact of the SNR value on prediction error and sparsity for Synthetic Dataset2.

6.6.4 Summary

Across both synthetic datasets, MIO and First-Order behave very similarly to each other in both prediction error and sparsity, suggesting that the computationally cheap First-Order heuristic achieves performance close to the exact MIO solver in this setting. The most consistent difference across all six experiments concerns sparsity control: Lasso reliably selects more variables than the true number of active variables, and this over-selection worsens as number of features p increases, whereas MIO and First-Order remain close to the true k_0 regardless of n , p , or SNR. Differences in prediction error are comparatively smaller, with Lasso falling most noticeably behind in the p -experiments and at low SNR.

6.7 Experiment 4

6.7.1 Setup

This experiment is conducted on expanded Diabetes dataset (see Section 6.2). Model evaluation is performed using $K = 5$ -fold cross-validation. Within each fold, the subset size k for MIO and for the First-Order heuristic is selected independently via an inner train/validation split (75%/25%) of that fold’s training data, by minimizing validation mean squared error over the candidate grid

$$k \in \{5, 10, 15, 20, 25, 30, 35, 40\}.$$

The experiment evaluates three methods (same as for the Experiment 6.6): MIO with warm-up, First order with hard thresholding, Lasso. The evaluation is based on the test-set mean squared error (MSE), the number of nonzero coefficients selected. Additionally, fitting time are recorded for each method. Each measure is recorded and averaged across the 5 folds.

6.7.2 Results

Figure 9 summarizes the comparison of MIO, the First-Order and Lasso on the expanded Diabetes dataset, averaged over the 5 cross-validation folds (error bars denote standard deviation across folds).

The three methods achieve nearly identical test MSE: 0.00118 for MIO, 0.00115 for First-Order, and 0.00113 for Lasso. The differences between methods are small which suggests that, on this dataset, none of the three approaches offers a clear predictive advantage over the others.

A much clearer separation emerges in model size. MIO selects on average 9 nonzero coefficients and First-Order selects 8, both close to each other and noticeably less than Lasso, which selects on average 15 nonzero coefficients, despite achieving comparable predictive accuracy. MIO also shows the largest fold-to-fold variability in selected sparsity (ranging roughly between 4 and 14 across folds), reflecting sensitivity of the validation-based k -selection to the specific train/validation split in this noisy dataset.

The three methods differ by several orders of magnitude in fitting time. MIO requires on average 225 seconds per fold (including the k -selection stage across all 8 candidate values), First-Order requires 3.65 seconds, and Lasso requires only 0.099 seconds. This reflects the fundamental computational gap between solving a mixed-integer program to near-optimality and the much cheaper heuristic and convex alternatives. First-Order achieves sparsity and accuracy comparable to MIO at a small fraction of the computational cost, making it an attractive practical alternative when an exact optimality certificate is not required.

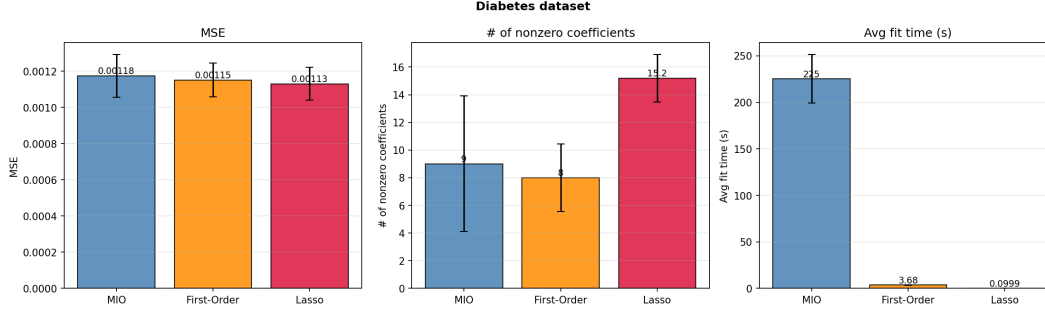


Figure 9: Performance results on Diabetes dataset.

7 Summary

This project compared three methods for sparse regression problem: Mixed-Integer Optimization (MIO), a first-order hard-thresholding method, and Lasso.

MIO finds the exact solution to the best subset selection problem, but it can be computationally expensive, especially when the number of variables is large or when predictors are highly correlated. The first-order method is much faster and can also be used to provide a good starting point for MIO, which helps the solver converge more quickly.

The experiments showed that MIO and the first-order method give very similar results. Both methods usually select a number of variables close to the true sparsity level k_0 . In contrast, Lasso tends to select many more variables than necessary, especially when the number of predictors or the correlation between predictors increases.

In terms of prediction accuracy, MIO usually achieved the lowest prediction error, although the first-order method often produced very similar results. Lasso generally performed worse when predictor correlation was high.

The main disadvantage of MIO is its running time. It is much slower than both the first-order method and Lasso, and the difference becomes very large when predictors are strongly correlated. The convergence study showed that most of MIO's improvement happens during the first few seconds of optimization and that warm-starting with the first-order solution improves convergence.

Overall, the results confirm that MIO provides the best sparsity control and very strong predictive performance, but at a high computational cost. The first-order method offers a good balance between accuracy and speed, while Lasso remains attractive because it is fast.

References

- [1] Bertsimas, D., King, A., Mazumder, R. (2016). *Best subset selection via a modern optimization lens*. Annals of Statistics.